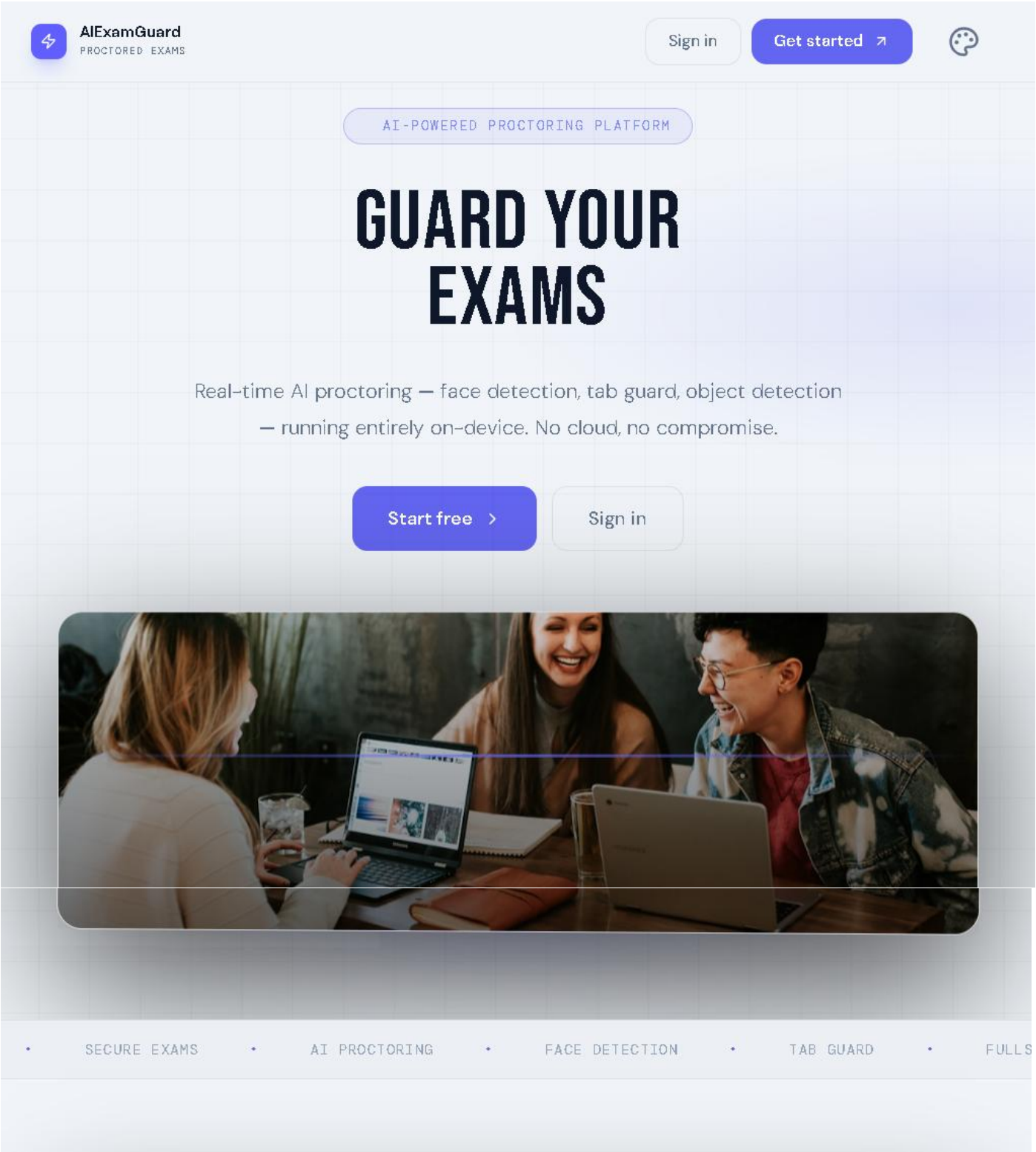
Images of the User Interface (Frontend)

1. Landing Page/Home Page of the Website



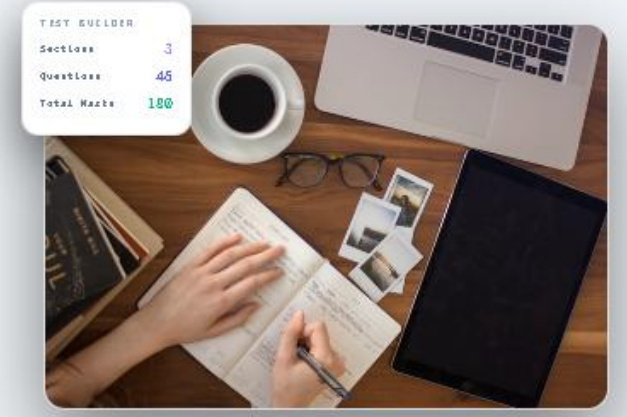
FOR STUDENTS

FOCUS ON THE EXAM

Clean, distraction-free interface. Navigate questions freely within each section, track your answers visually, and submit with confidence. The AI handles security — you handle the answers.

- Section-wise timed questions with live countdown
- Visual question map — see answered at a glance
- MCQ and numerical answer types supported
- One-click submit with instant confirmation

Register as Student



FOR CREATORS

BUILD TESTS IN MINUTES

A three-panel editor that gets out of your way. Add sections, set timers, write MCQ or numerical questions, attach images, and marks auto-calculate. Publish with one click.

- Unlimited sections with individual timers
- MCQ with images + numerical answer types
- Auto marks calculation across all sections
- Publish, draft, edit and delete anytime

Start creating

UNDER THE HOOD

BUILT FOR INTEGRITY

Face Tracking

Continuous face detection flags absence or multiple people in real time.

Device Detection

On-device AI spots phones, books and prohibited items without cloud upload.

Fullscreen Lock

Enforced fullscreen with instant re-entry prompts and violation logging.

Tab Guard

Every tab switch is caught, logged, and triggers an immediate warning.

Section Timers

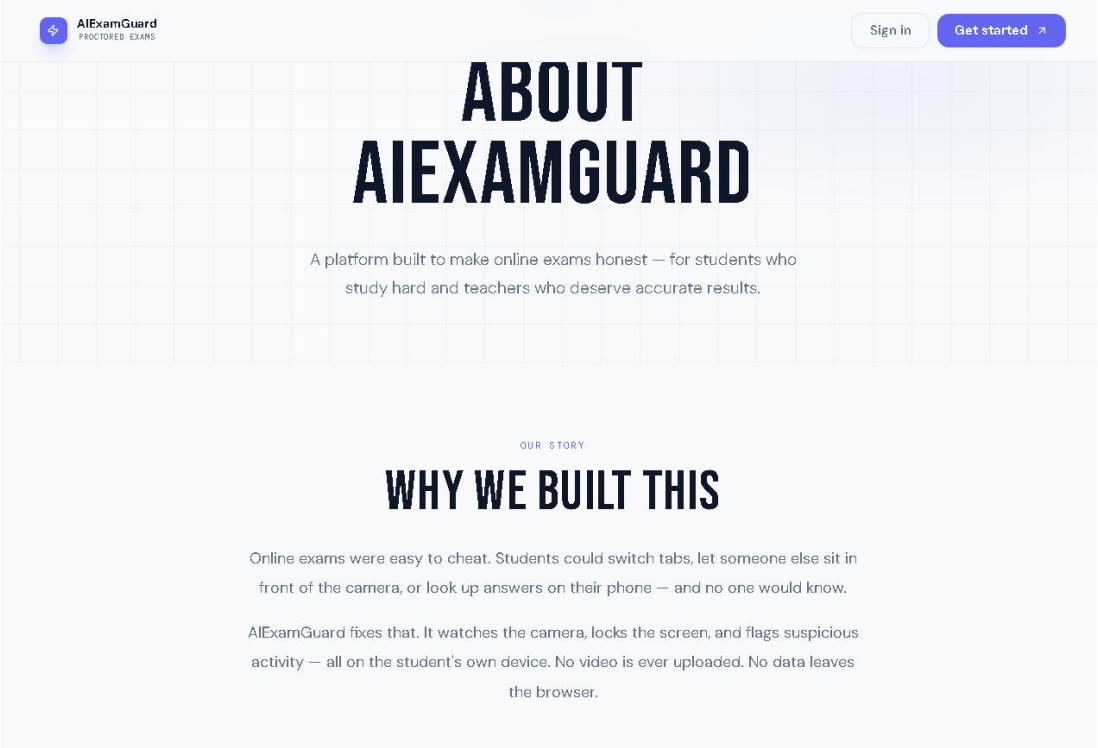
Per-section countdowns auto-advance students — fair for everyone.

Private by Design

Zero video upload. All AI runs locally. Your data never leaves the device.

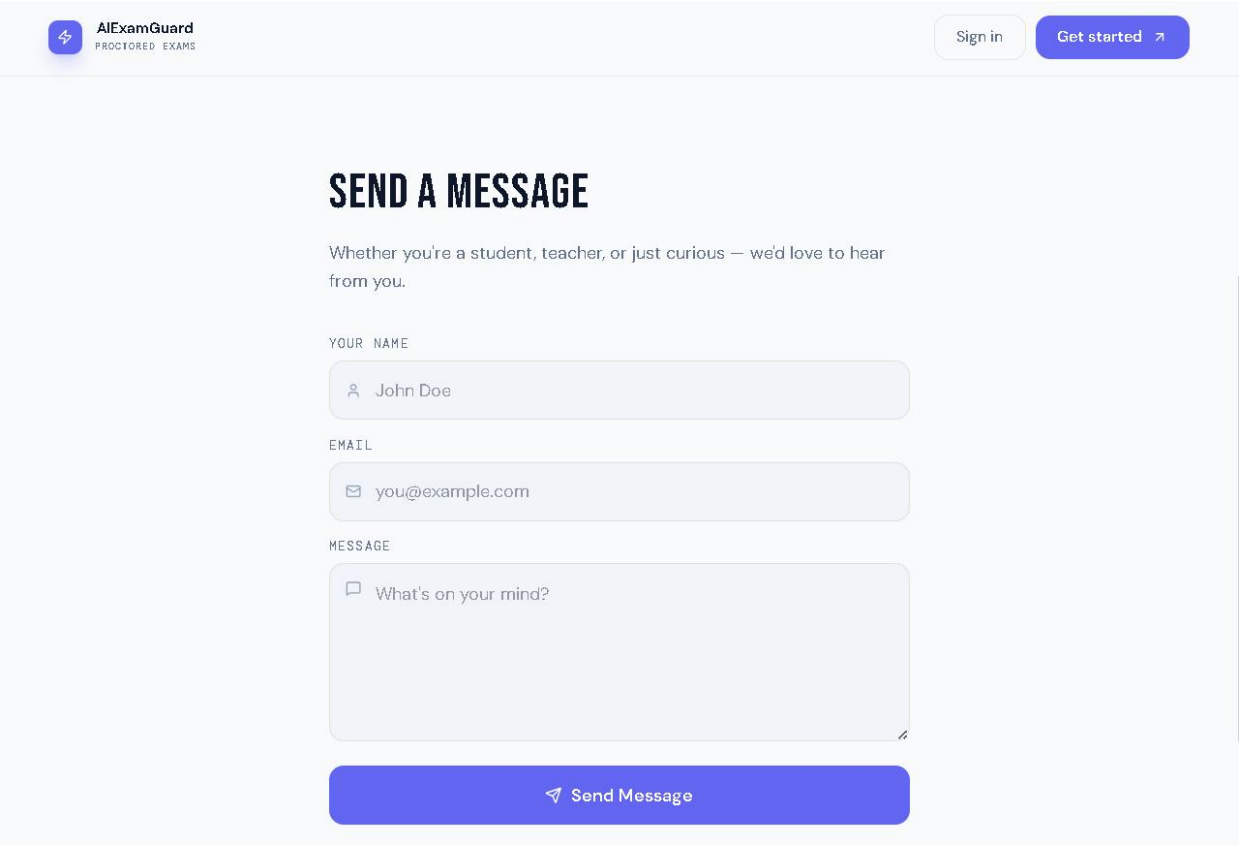
2. About Page

It’s a basic about page of the website telling its story ,goal and mission.



3. Contact Us Page

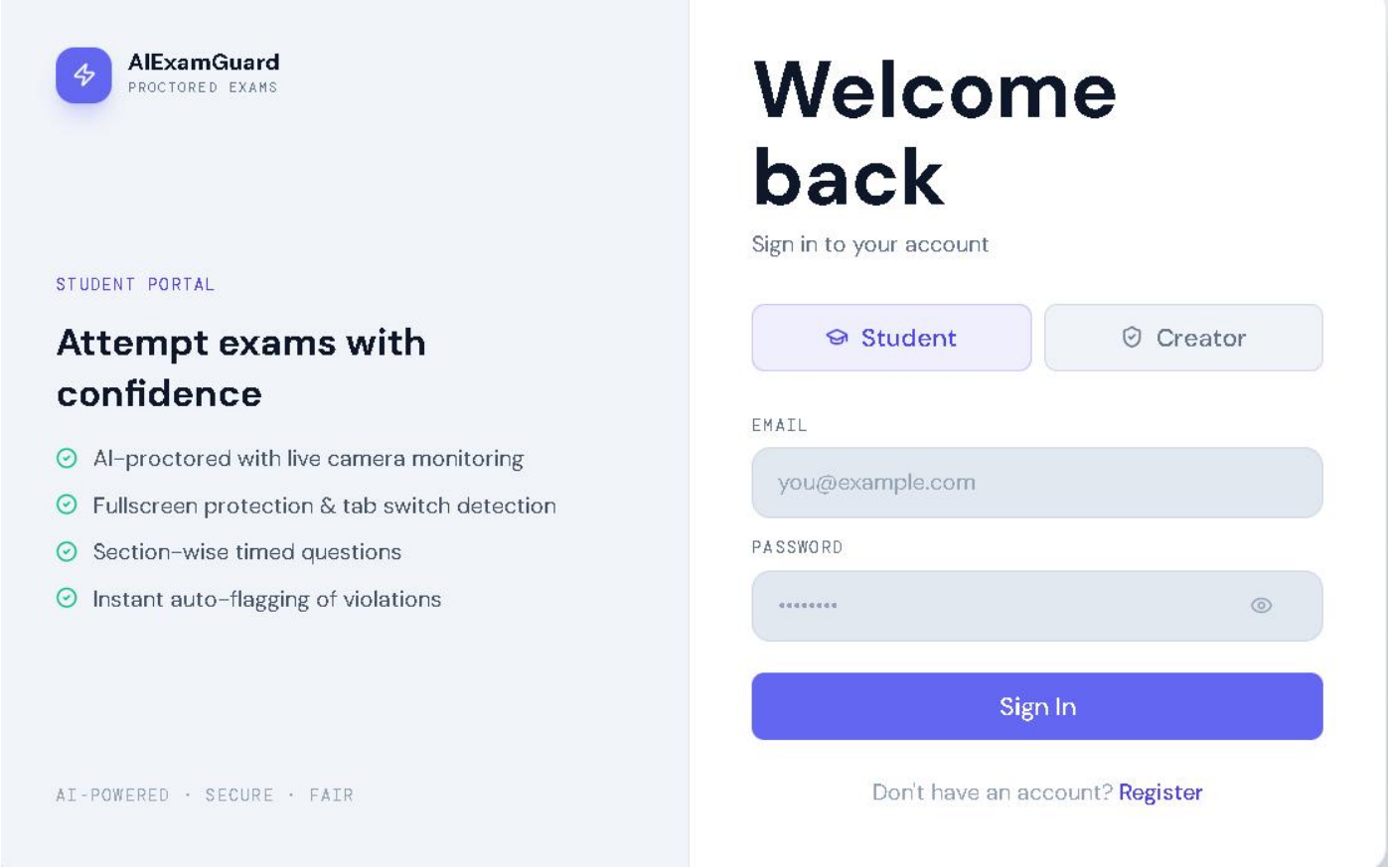
It’s a basic contact page of me from which users can contact me through email .



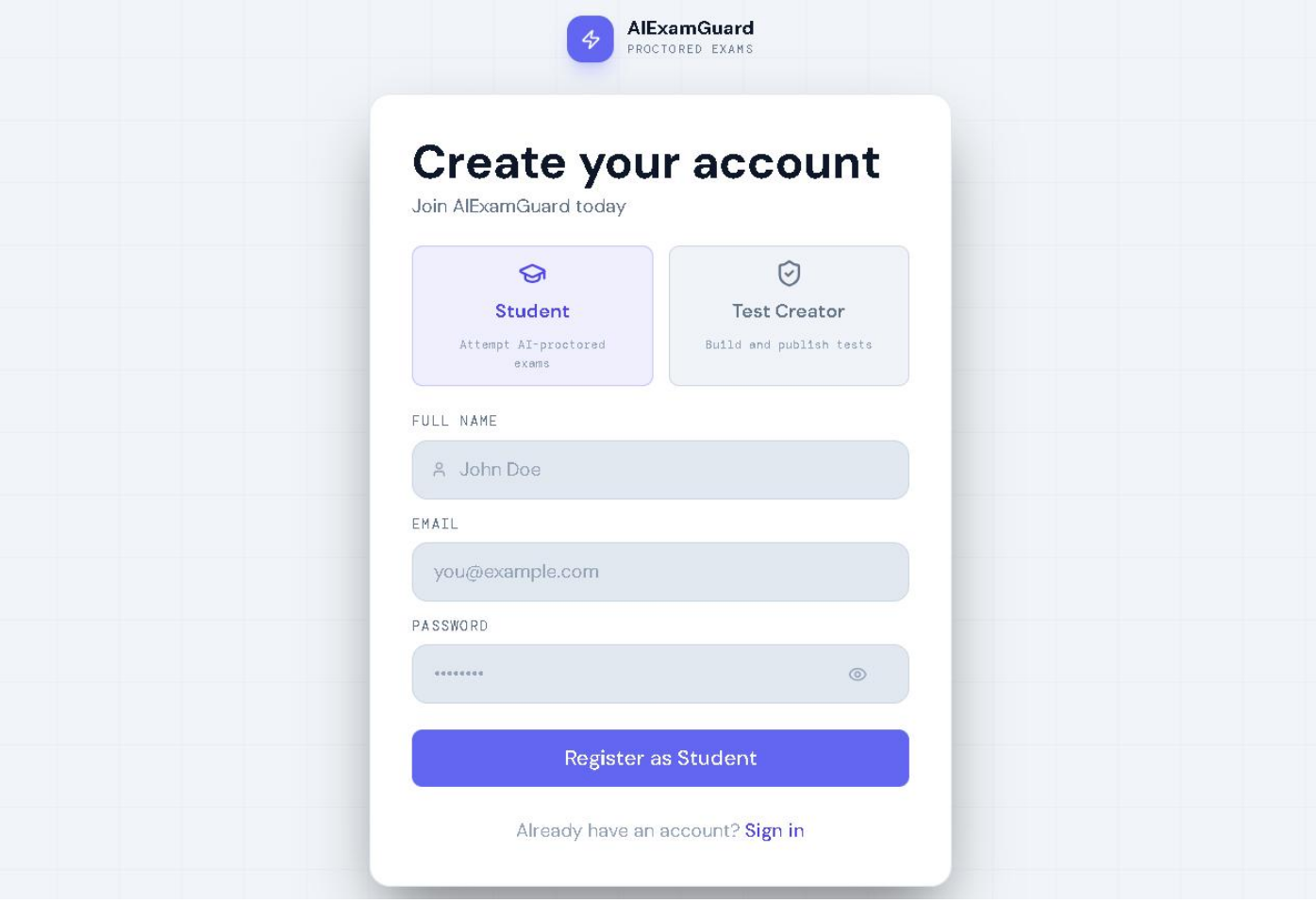
4.Login /Sign-Up - Every website needs to maintain info about their users for better user experience and to identify users to do which uneeds to sign up and then can log in to website

-

Login



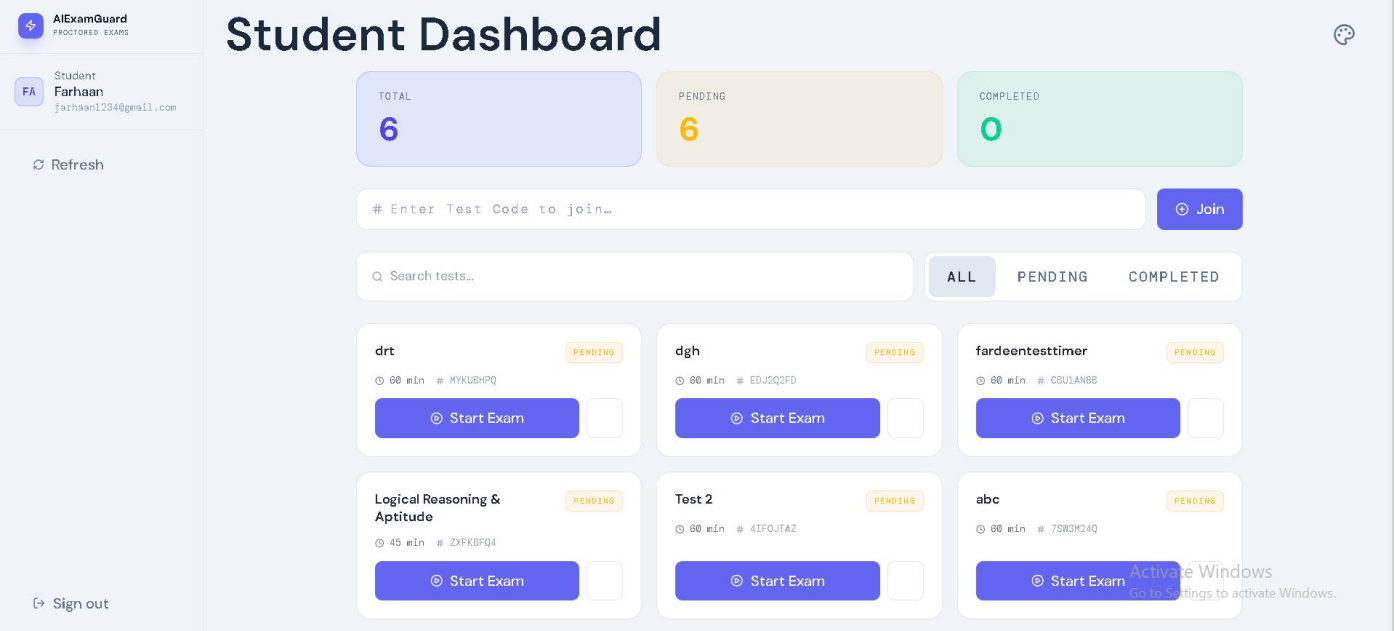
Sign Up


AIExamGuard
PROCTORED EXAMS
Create your account
Join AIExamGuard today
Student
Attempt AI-proctored exams
Test Creator
Build and publish tests
FULL NAME
John Doe
EMAIL
you@example.com
PASSWORD
Register as Student
Already have an account? Sign in

5.Dashboards - This is the main page from which a student can join test and creator can manage tests and students

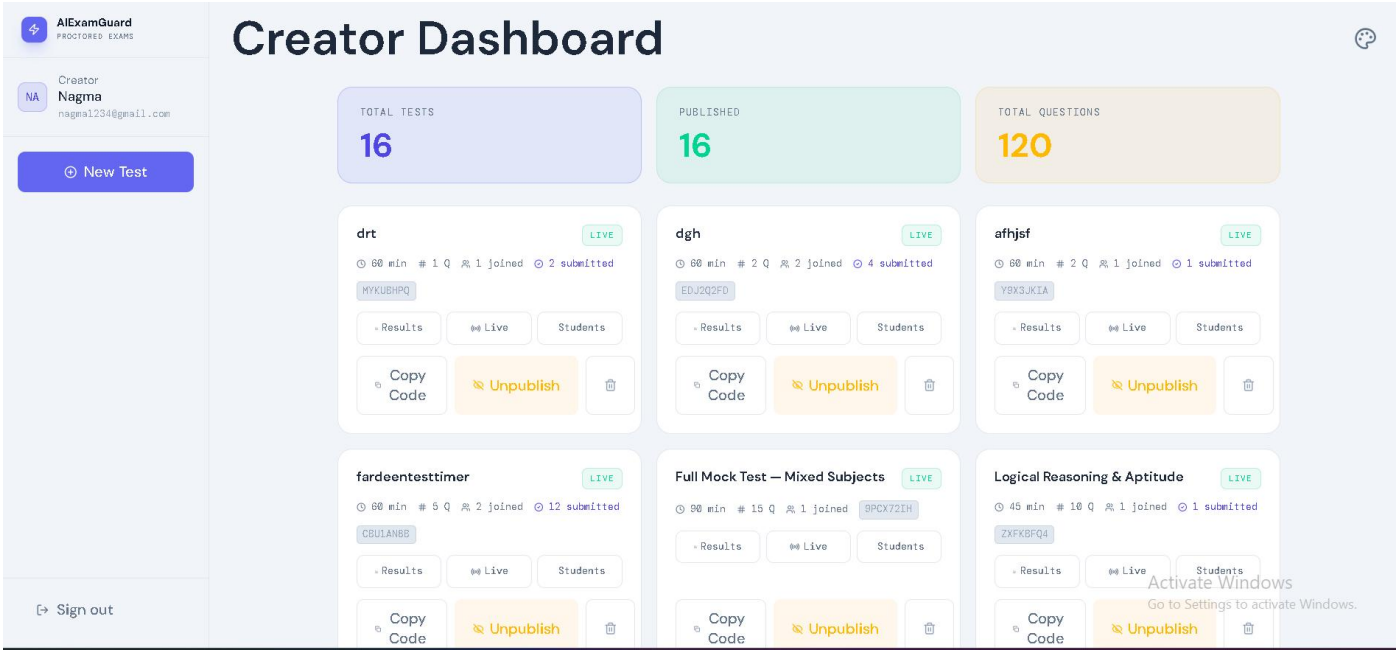
Student Dashboard Page

In student dashboard student can join tests through test shared by creator and can start attempting the test by clicking start exam



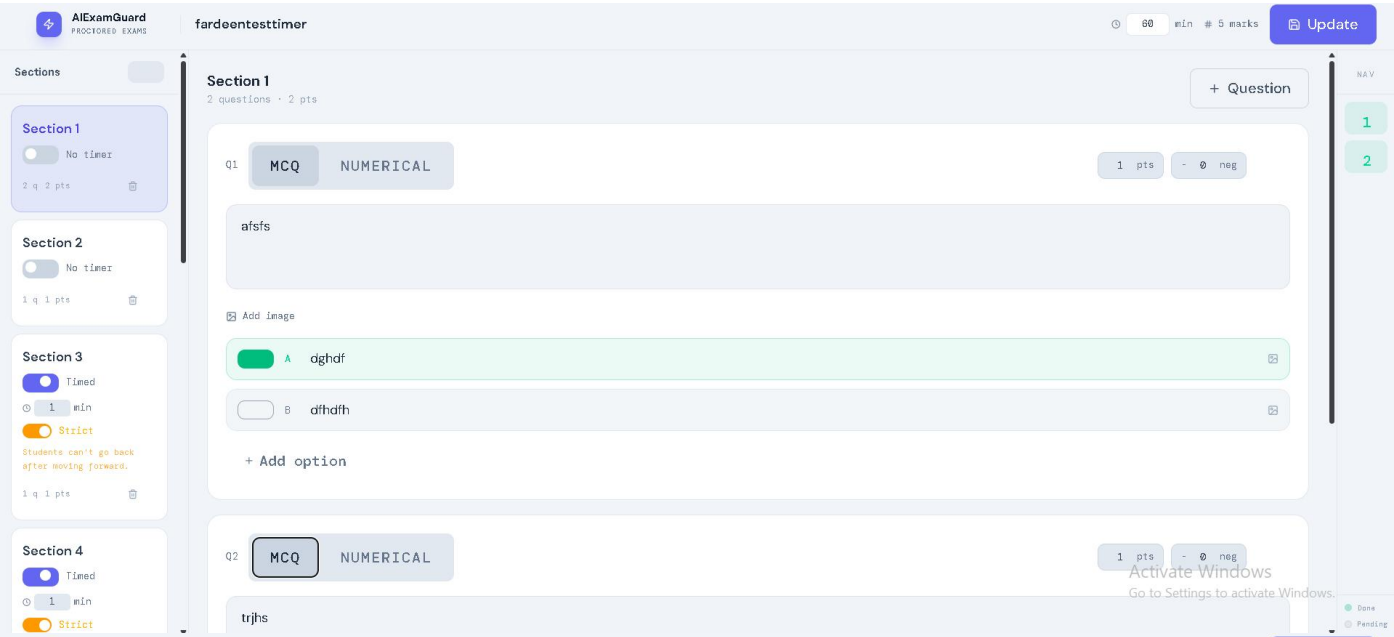
Creator Dashboard Page

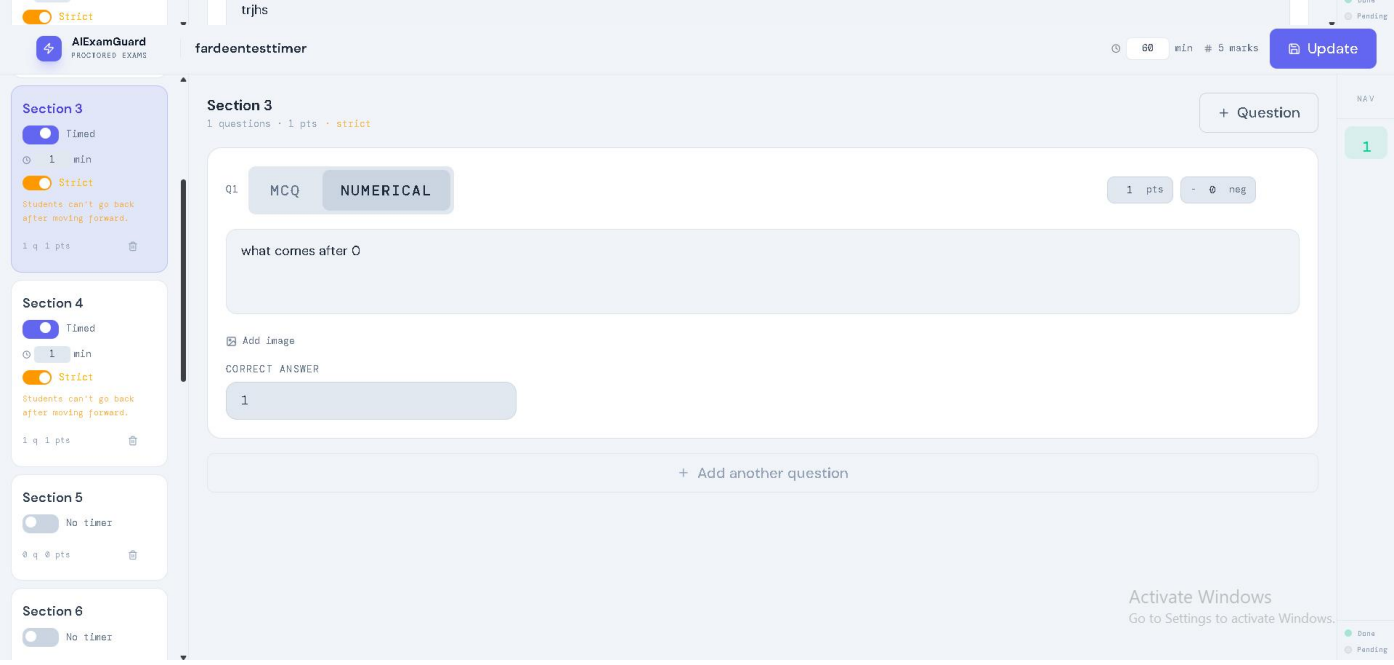
In creator dashboard creator can create tests new test by clicking new Test button or can update existing test by clicking upon the test itself also creator can see who attempted test,their live monitoring and their result



6.Test Creation Page -

This page lets the user create tests and add infinite no of questions and sections , each question can be of two types mcq and numerical type creator could also add image in questions as well as in mcq options .

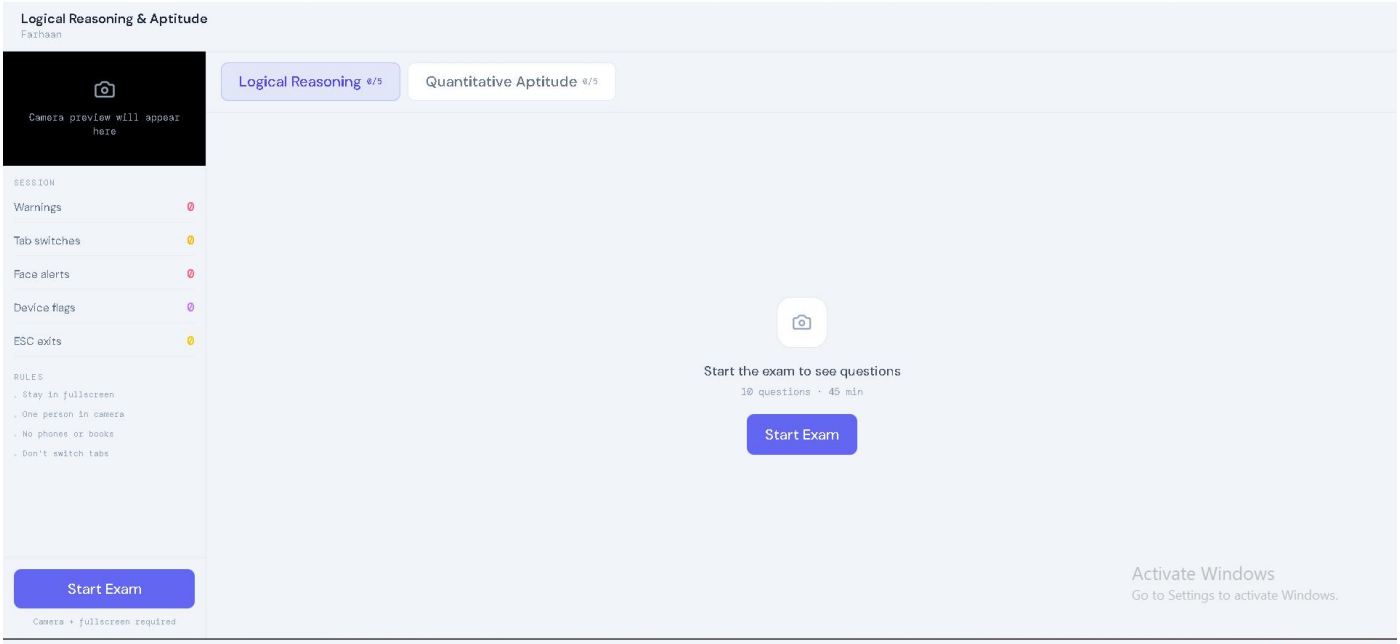


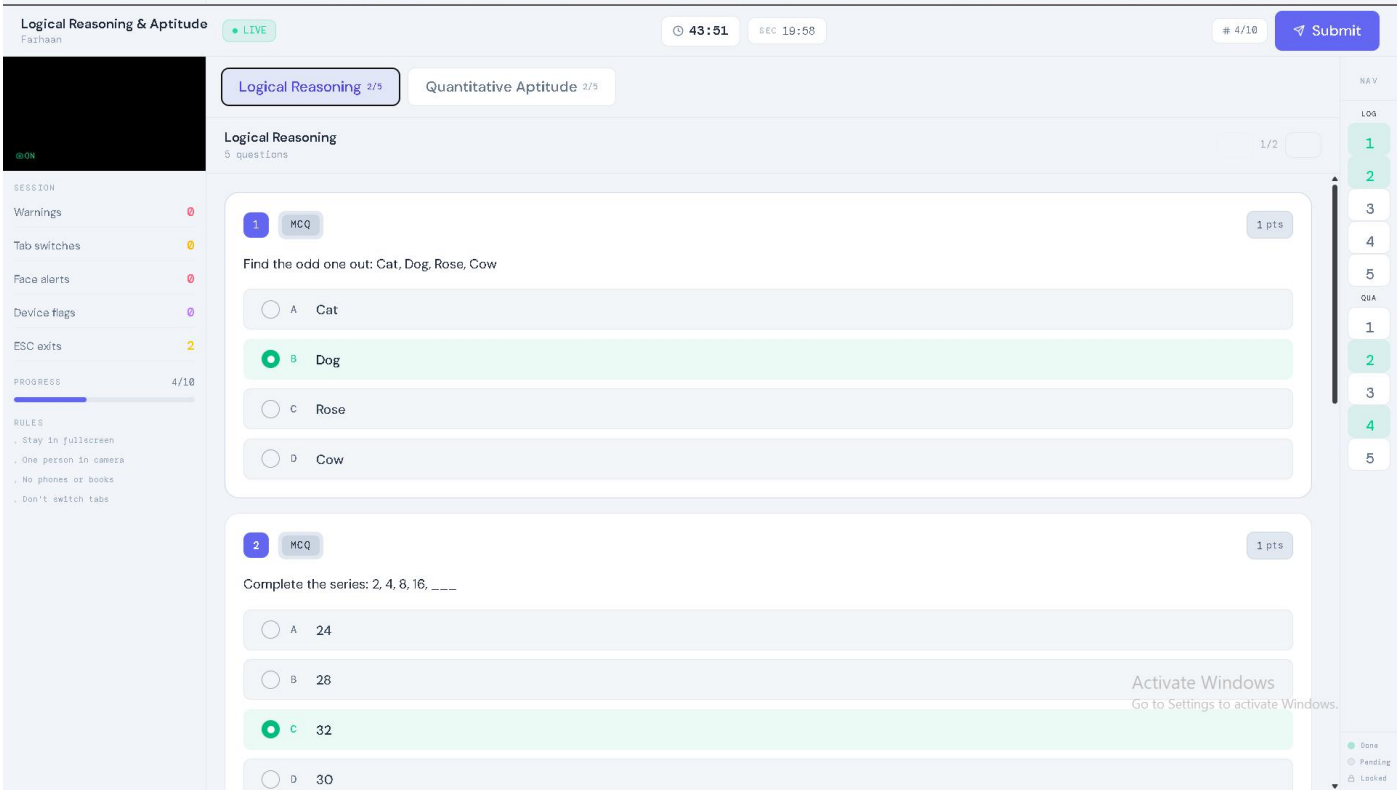


Creator can edit test title , creator can edit sections and can set timers for each sections as well creator could lock certain sections so that user cannot switch sections after coming to locked sections.

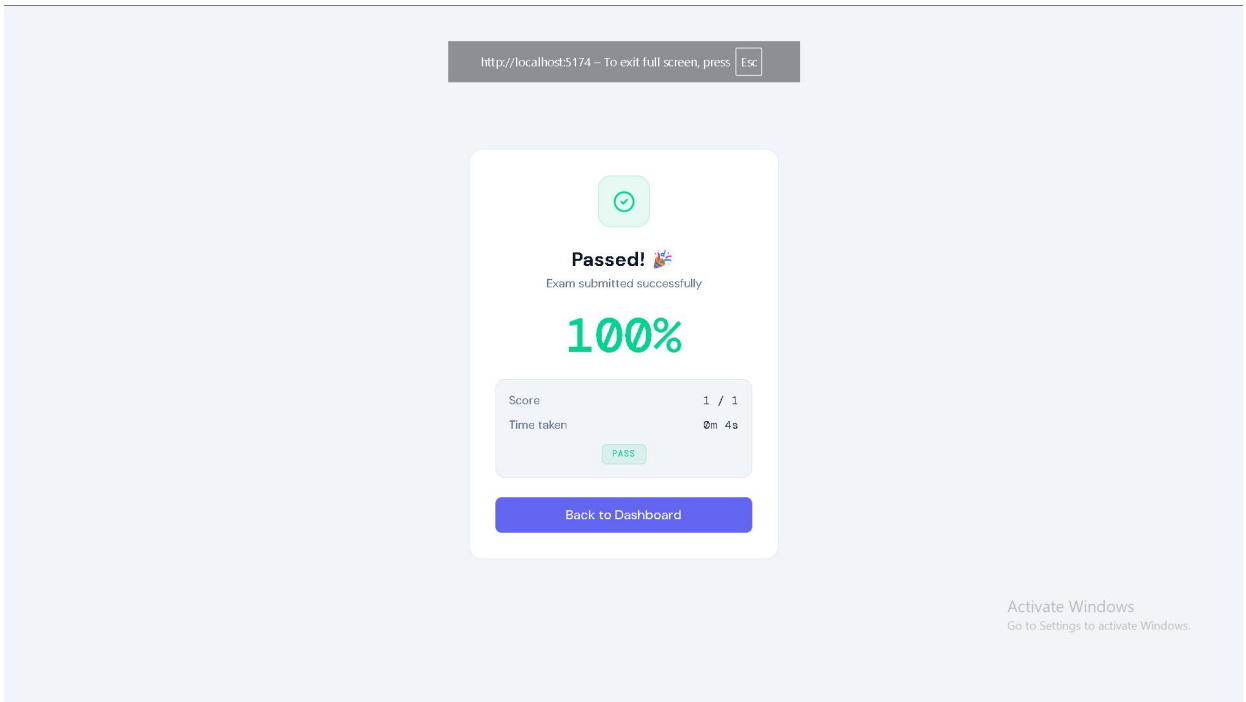
7.Exam Page -

This page is the backbone of the entire system upon which student spends most of their time by attempting test generated by the creators .

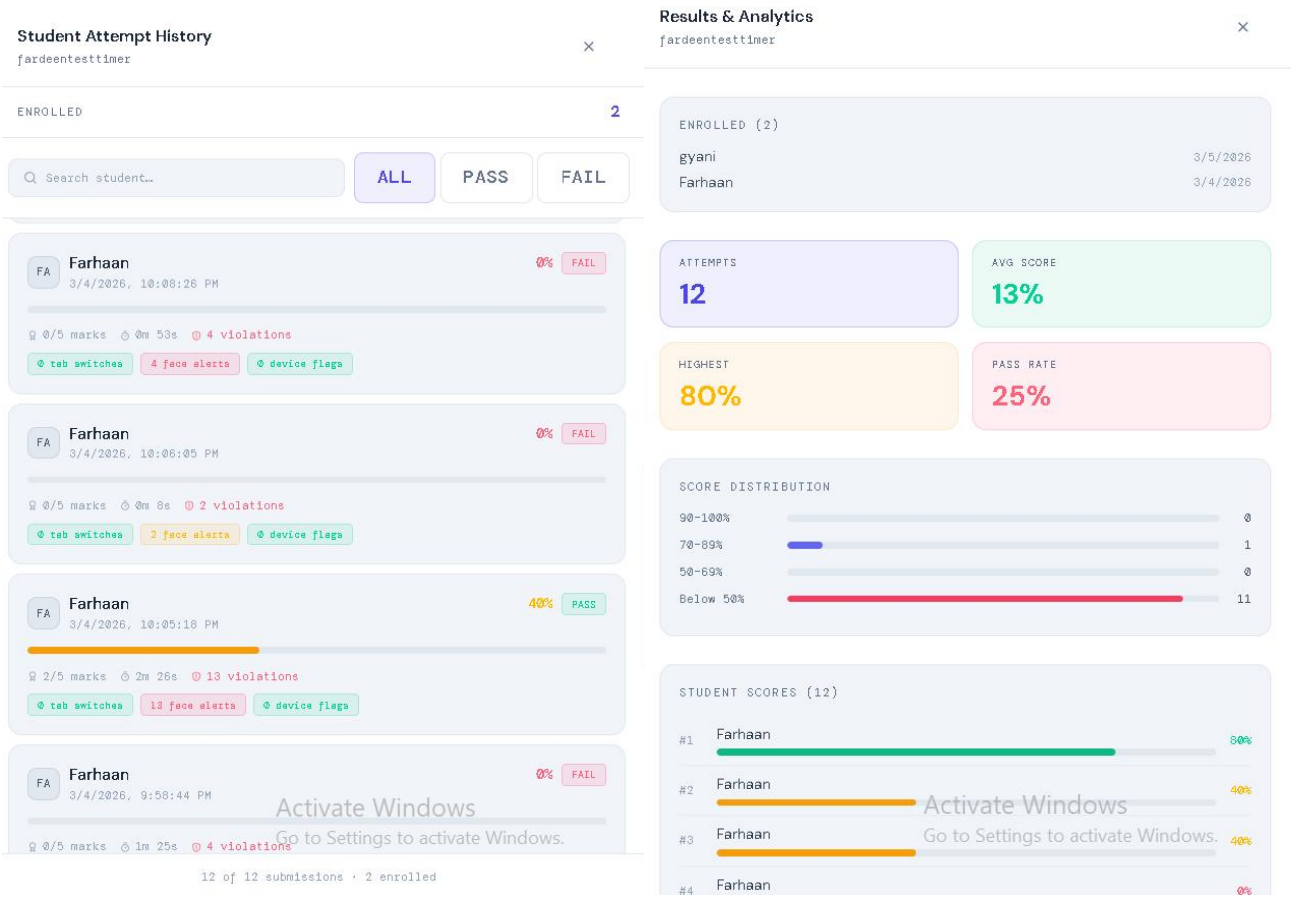




Result Pop-Up after Submitting the Test

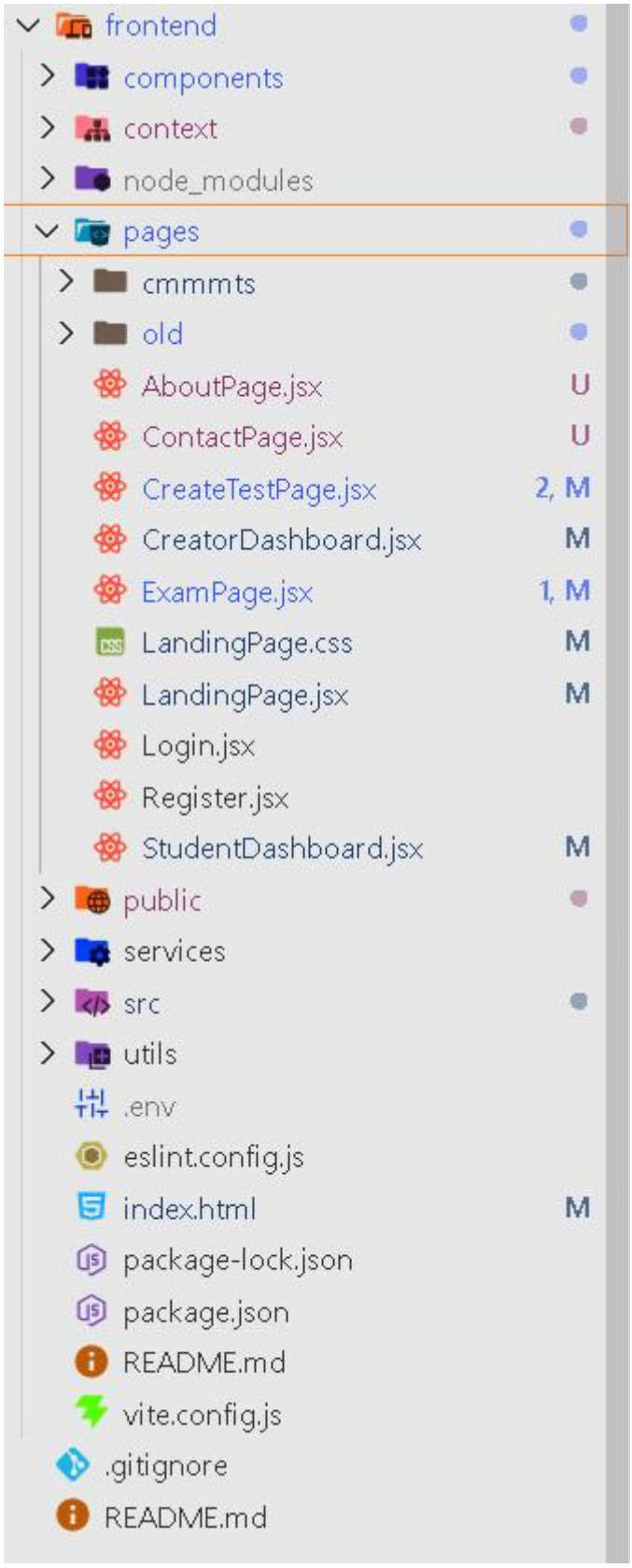


8. Result and Attempts Page (Only For Creator to see)



Code for Frontend

1. Frontend Project Structure



Main -- App.jsx

frontend > src > App.jsx > App

```jsx
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import ProtectedRoute from "../components/ProtectedRoute";
import LandingPage from "../pages/LandingPage";
import Login from "../pages/Login";
import Register from "../pages/Register";
import AboutPage from "../pages/AboutPage";
import ContactPage from "../pages/ContactPage";
import CreatorDashboard from "../pages/CreatorDashboard";
import StudentDashboard from "../pages/StudentDashboard";
import ExamPage from "../pages/ExamPage";
import CreateTestPage from "../pages/CreateTestPage";

export default function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          <Route path="/" element={<LandingPage />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/about" element={<AboutPage />} />
          <Route path="/contact" element={<ContactPage />} />
          <Route path="/creator-dashboard" element={<ProtectedRoute role="creator"><CreatorDashboard /></ProtectedRoute>} />
          <Route path="/create-test" element={<ProtectedRoute role="creator"><CreateTestPage /></ProtectedRoute>} />
          <Route path="/create-test/:testId" element={<ProtectedRoute role="creator"><CreateTestPage /></ProtectedRoute>} />
          <Route path="/student-dashboard" element={<ProtectedRoute role="student"><StudentDashboard /></ProtectedRoute>} />
          <Route path="/exam/:id" element={<ProtectedRoute role="student"><ExamPage /></ProtectedRoute>} />
          <Route path="*" element={<Navigate to="/login" replace />} />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}
```

Main.jsx

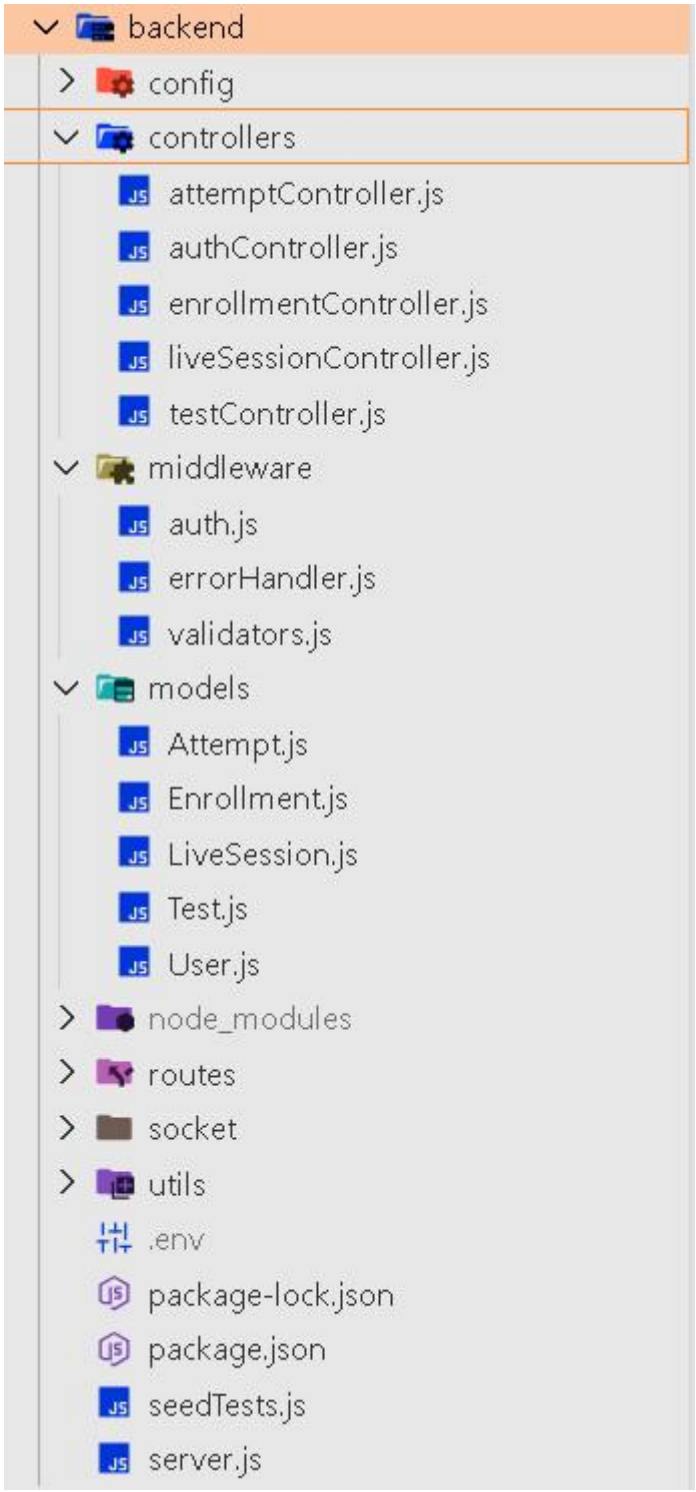
frontend > src > main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import { ThemeProvider } from "../context/ThemeContext";
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <ThemeProvider>
    <StrictMode>
      <App />
    </StrictMode>
  </ThemeProvider>
)
```

Complete Backend

1. Project Structure of Backend



2. Server.js - Entry point for all the request to backend

backend > server.js > ...

```javascript
require("node:dns/promises").setDefaultResultOrder("ipv4first");
require("node:dns/promises").setServers(["8.8.8.8", "1.1.1.1", "8.8.4.4"]);
require("dotenv").config();
const express      = require("express");
const cors         = require("cors");
const connectDB    = require("./config/db");
const errorHandler = require("./middleware/errorHandler");
connectDB();
const app = express();
app.use(cors({
  origin:      [process.env.CLIENT_URL || "http://localhost:5173", "http://localhost:5174"],
  credentials: true,
}));
app.use(express.json({ limit: "10mb" }));
app.use(express.urlencoded({ extended: true }));
app.get("/api/health", (req, res) => {
  res.json({
    success: true,
    message: "AIExamGuard API is running",
    env:     process.env.NODE_ENV,
    time:    new Date().toISOString(),
  });
});
app.use("/api/auth",        require("./routes/authRoutes"));
app.use("/api/tests",       require("./routes/testRoutes"));
app.use("/api/enrollments", require("./routes/enrollmentRoutes"));
app.use("/api/attempts",    require("./routes/attemptRoutes"));
app.use((req, res) => {
  res.status(404).json({ success: false, message: `Route ${req.originalUrl} not found` });
});
app.use(errorHandler);
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`🚀  Server running in ${process.env.NODE_ENV} mode on port ${PORT}`);
});

module.exports = app;
```

Activate Windows
Go to Settings to activate Windows.

3. MongoDB Database

MongoDB Compass - suzane.fploc.mongodb.net/CBT_platform.tests

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

- suzane.fploc.mongodb.net
  - CBT_platform
    - attempts
    - enrollments
    - livesessions
    - tests
    - users
  - Ecommerce
  - User_Authentication
  - admin
  - image_uploader
  - local
  - sample_mflix
  - test
  - urlShortener

tests

suzane.fploc.mongodb.net > CBT_platform > tests

Open MongoDB shell

Documents 16 | Aggregations | Schema | Indexes 2 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | UPDATE | DELETE | EXPORT DATA | EXPORT CODE

25 | 1 - 16 of 16

```
_id: ObjectId('69a8321a6b01d6db761a00e9')
title : "Computer Science — Basics"
description : "Covers data structures, algorithms, networking and OS concepts."
createdBy : ObjectId('69a58fbda3c050e6e5868dc0')
duration : 50
sections : Array (2)
  0: Object
    title : "Programming & Data Structures"
    duration : 1500
    questions : Array (5)
      0: Object
        type : "mcq"
        text : "Which data structure uses LIFO?"
        image : null
        options : Array (4)
        correct : "Stack"
        marks : 1
        negativeMark : 0.25
        _id : ObjectId('69a8321a6b01d6db761a00eb')
      1: Object
      2: Object
      3: Object
      4: Object
    _id : ObjectId('69a8321a6b01d6db761a00ea')
  1: Object
published : true
settings : Object
createdAt : 2026-03-04T13:22:34.205+00:00
```

Activate Windows
Go to Settings to activate Windows.